

Input & Output Statements

- 1. What is the purpose of input/output functions in C programming**
 - a. To perform arithmetic operations
 - b. To manage memory allocation
 - c. To facilitate interaction between the user and the program
 - d. To define the structure of a program
- 2. Which library is required to use I/O functions in C**
 - a. <stdlib.h>
 - b. <conio.h>
 - c. <stdio.h>
 - d. <string.h>
- 3. What is the role of the printf() function in C**
 - a. To take input from the user
 - b. To print output to the console
 - c. To declare variables
 - d. To execute loops
- 4. What is the purpose of the scanf() function in C**
 - a. To read data from the user
 - b. To display output
 - c. To perform mathematical calculations
 - d. To allocate memory
- 5. What is the difference between formatted and unformatted I/O functions**
 - a. Formatted I/O uses headers, unformatted does not
 - b. Formatted I/O processes structured data, unformatted handles raw data
 - c. Formatted I/O works faster than unformatted
 - d. Unformatted I/O uses special characters
- 6. What is a format specifier in C programming**
 - a. A variable declaration tool
 - b. A code for representing data types in I/O functions
 - c. A function to manage memory
 - d. A way to write conditional statements

7. Which format specifier is used for integers in C

- a. %d
- b. %f
- c. %c
- d. %s

8. What is the purpose of the address operator in scanf()

- a. To allocate memory dynamically
- b. To provide the memory address of a variable
- c. To specify the type of data
- d. To read characters from the user

9. What is a precision specifier in C

- a. A way to allocate memory
- b. A method for precise calculations
- c. A control for output formatting
- d. A specifier for handling errors

10. What happens if the wrong format specifier is used in printf()

- a. The program crashes
- b. The output is incorrect or undefined
- c. The compiler skips the line
- d. The function terminates execution

11. What is an example of an invalid format specifier

- a. %z
- b. %d
- c. %f
- d. %c

12. How does the %s specifier handle strings in printf()

- a. Prints the string with a newline
- b. Prints the string until a null character is encountered
- c. Trims spaces from the string
- d. Reverses the string

13. Which format specifier is used to print floating-point numbers in C

- a. %c
- b. %d
- c. %s
- d. %f

14. What does the %c format specifier represent

- a. A string
- b. A single character
- c. An integer
- d. A floating-point number

15. How can we control the number of decimal places in printf()

- a. By using a format specifier like %.nf
- b. By specifying precision in scanf()
- c. By adding additional parameters in printf()
- d. By using the address operator

16. Which format specifier is used to display hexadecimal numbers

- a. %x
- b. %d
- c. %f
- d. %c

17. What is the output of the following C code?

```
#include <stdio.h>
```

```
int main() {  
    float num = 3.14159;  
    printf("%.2f", num);  
    return 0;  
}
```

- a. 3.14
- b. 3.14159
- c. 3.1
- d. 3

18. Which format specifier is used to display octal numbers

- a. %o
- b. %d
- c. %f
- d. %c

19. What is the correct syntax for printing an integer using printf?

- a. `printf("%d", num);`
- b. `printf("%i", num);`
- c. `printf("%f", num);`
- d. `printf("%s", num);`

20. How would you read an integer value from the user using scanf?

- a. `scanf("%i", &num);`
- b. `scanf("%d", num);`
- c. `scanf("%d", &num);`
- d. `scanf("%f", &num);`